CCIT Report #576
March 2006

Timeliness, Failure-Detectors, and Consensus Performance

Idit Keidar* Alexander Shraer†

Abstract

We study the implication that various timeliness and failure detector assumptions have on the performance of consensus algorithms that exploit them. We present a general framework, GIRAF, for expressing such assumptions, and reasoning about the performance of indulgent algorithms.

Keywords: eventual synchrony, failure detectors, lower bounds, indulgent consensus.

1 Introduction

1.1 Background and motivation

Consensus is a widely-studied fundamental problem in distributed computing, theory and practice. Roughly speaking, it allows processes to agree on a common output. We are interested in the performance of consensus algorithms in different timing models.

Although the synchronous model provides a convenient programming framework, it is often too restrictive, as it requires implementations to use very conservative timeouts to ensure that messages are *never* late. For example, in some practical settings, there is a difference of two orders of magnitude between average and maximum message latencies [6, 5]. Therefore, a system design that does not rely on strict synchrony is often advocated [25, 17, 8]; algorithms that tolerate arbitrary periods of asynchrony are called *indulgent* [20].

As it is well-known that consensus is not solvable in asynchronous systems [18], the feasibility of indulgent consensus is contingent on additional assumptions. More specifically, such a system may be asynchronous for an unbounded period of time, but eventually reaches a *Global Stabilization Time (GST)* [17], following which certain properties hold. These properties can be expressed in terms of eventual timeliness of communication links [17, 10][1], or using the abstraction of *oracle failure detectors* [8]. Protocols in such models usually progress in asynchronous *rounds*, where, in each round, a process sends messages (often to all processes), then receives messages while waiting for some condition expressed as a timeout or as the oracle's output, and finally performs local processing.

Recent work has focused on weakening post-GST synchrony assumptions [22, 1, 2, 3, 27], e.g., by only requiring one process to have timely communication with other processes after GST. Clearly, weakening timeliness requirements is desirable, as this makes it easier to meet them. For example, given a good choice of a leader process, it is possible to choose a fairly small timeout, so that the leader is almost always able to communicate with all processes before the timeout expires, whereas having each process usually succeed to communicate with *every* other processes requires a much larger timeout [5, 4]. In general, the weaker the eventual timeliness properties assumed by an algorithm are, the shorter the timeouts its implementation needs to use, and the faster its communication rounds can be.

*Department of Electrical Engineering, Technion, Israel. idish@ee.technion.ac.il.
†Department of Computer Science, Technion, Israel. shralex@cs.technion.ac.il.

[1] A timely link delivers messages with a bounded latency; the bound is either known or unknown a priori.